FirmApp - Preguntas y respuestas de defensa

TINF1119 Desarrollo Móvil - Evaluación N°3. Respuestas resumidas; en cursiva naranja, dónde está en el código.

1. Arquitectura y Kivy

P: ¿Qué es FirmApp y qué necesidad resuelve?

Una app móvil (Kivy/KivyMD) para que un trabajador firme documentos de RR.HH. (liquidaciones, contratos) con 1 toque. Responde a la encuesta de diagnóstico: 72,7 % exige firmar 'de un toque' desde el celular y 54,6 % califica de difícil su sistema actual.

Ver: README.md, FUNDAMENTACION-UX-UI.md

P: ¿Cómo navega entre pantallas?

Con un **ScreenManager** y transición **FadeTransition**. Hay 5 pantallas (login, register, home, sign, success), creadas una sola vez en **InterfazRRHHApp.build()**. Navegar solo cambia **sm.current**; no se crean ni destruyen widgets.

Ver: main.py: build(), go_to_register(), go_to_home_after_login(), go_to_sign(), go_to_success(), go_home()

P: ¿Qué layouts usaste?

MDBoxLayout para apilar elementos (vertical/horizontal), **MDFloatLayout** cuando hay que superponer (placeholder sobre un campo, overlays) y **MDCard** para las tarjetas. BoxLayout reparte el ancho entre sus hijos; FloatLayout los superpone - confundirlos causó un bug real en el login.

Ver: interfazrrhh.kv

P: ¿Por qué separar .kv y .py?

Arquitectura de la Sesión 12: la interfaz (estilos, posiciones) vive en **interfazrrhh.kv** y la lógica (validar, navegar, guardar) en Python. Kivy carga el .kv solo porque la clase se llama **InterfazRRHHApp** y el archivo **interfazrrhh.kv**.

Ver: main.py (docstring inicial), interfazrrhh.kv

P: ¿Qué diferencia hay entre un atributo Python y una Property de Kivy?

Solo las **Property** (StringProperty, BooleanProperty, NumericProperty) avisan al .kv cuando cambian, y la UI se repinta sola. Un atributo normal cambia pero la pantalla no se entera. Ejemplo: **valid** y **kind** en LoginIdentifierInput; **pending_count** en RootScreen.

Ver: widgets.py, main.py

2. Autenticación (registro y login)

P: ¿Cómo validas un RUT?

Con el algoritmo **módulo 11**: se multiplican los dígitos del cuerpo de derecha a izquierda por 2,3,4,5,6,7 (cíclico), se suman, y el dígito verificador es 11 - (suma mod 11) (10 = 'K', 11 = '0'). Se compara con el dígito ingresado.

Ver: validators.py: rut_check_digit(), is_valid_rut()

P: ¿Cómo validas el correo?

Con una expresión regular (EMAIL_RE) que exige formato **nombre@dominio.ext**. **identify(texto)** decide si lo escrito es RUT, correo o ninguno.

Ver: validators.py: is_valid_email(), identify()

P: ¿Cómo funcionan el registro y el login?

Registro: RegisterScreen valida identificador, contraseña y confirmación; llama a **DataManager.register_user()**, que rechaza duplicados y guarda la cuenta. **Login**: LoginScreen.login() llama a **DataManager.authenticate()**; si coincide navega al Inicio, si no muestra un mensaje de error.

Ver: main.py: LoginScreen, RegisterScreen; data_manager.py

P: ¿Cómo guardas las contraseñas?

Nunca en texto plano: se guarda el **hash SHA-256** (hashlib) en data/usuarios.json. Al iniciar sesión se hashea lo ingresado y se compara. Limitación honesta: faltaría una 'sal' por usuario para producción.

Ver: data_manager.py: _hash_password()

P: ¿Por qué no podía escribir un correo al inicio?

El autoformateo de RUT borraba cualquier carácter que no fuera dígito o K. Se corrigió aplicando el formato solo si todo lo escrito *podría* ser un RUT.

Ver: widgets.py: LoginIdentifierInput.set_text()

P: ¿Hay una cuenta para probar?

Sí: **demo@firmapp.cl / demo1234**. Se crea sola la primera vez que se abre la app, para poder hacer la demo sin registrarse.

Ver: data_manager.py: DataManager.__init__()

3. Persistencia en JSON

P: ¿Dónde se guardan los datos?

En la carpeta **data/**: **usuarios.json** (cuentas) y **documentos.json** (qué está pendiente y qué ya se firmó). Se crea sola al correr la app y está en .gitignore porque es dato de ejecución, distinto en cada equipo.

Ver: data_manager.py: DATA_DIR, USERS_FILE, DOCS_FILE; .gitignore

P: ¿Cuándo se guarda y cuándo se lee?

Se **lee** al iniciar (main.py: PENDING, RECENT = DM.load_documents(...)). Se **guarda** cada vez que se firma: **complete_signature()** mueve el documento de PENDING a RECENT y llama a save_documents().

Ver: main.py: complete_signature(); data_manager.py

P: ¿Cómo compruebas que persiste?

Firmo un documento, cierro la app y la abro de nuevo: ya no aparece como pendiente, está en 'Recientes'. Lo verifiqué con dos procesos Python separados.

Ver: README.md: Persistencia de datos (JSON)

P: ¿Qué pasa si el archivo no existe o está corrupto?

Cada lectura/escritura va en **try/except** (FileNotFoundError, IOError/OSError, JSONDecodeError). Si falla, se usan los datos de ejemplo o se ignora ese guardado: un error de disco nunca tumba la app.

Ver: data_manager.py

4. Notificaciones

P: ¿Cómo implementaste las notificaciones?

Con **plyer.notification.notify()**, que usa la notificación nativa del sistema operativo con el mismo código Python. Está envuelta en try/except para que un fallo del SO no interrumpa el flujo.

Ver: notifications.py: notify()

P: ¿Qué evento dispara la notificación?

Dos eventos: al abrir la app si hay un documento **urgent** pendiente, y al **firmar con éxito** (go_to_success()).

Ver: main.py

P: ¿Para qué sirve la campana y el banner?

La notificación del SO es puntual y desaparece. La campana (y el banner naranja de Remote Config) abre **NotificationOverlay**, un aviso grande dentro de la app con el mensaje y la cantidad de documentos urgentes; se cierra tocando fuera.

Ver: main.py: NotificationOverlay, show_notifications(), hide_notifications()

5. Remote Config

P: ¿Qué es Remote Config y qué valores usas?

Cambiar el comportamiento de la app sin reinstalarla, leyendo un JSON externo. Tres parámetros: **banner_message** (texto), **show_desktop_cta** (feature flag que muestra/oculta una tarjeta) y **max_recent_docs** (límite numérico).

Ver: remote/config.json; remote_config.py: DEFAULTS

P: ¿Cuál es el flujo seguro?

Default local (la app arranca completa sin red) -> **Fetch asíncrono** con UrlRequest (no congela la UI) -> **Activate** si el JSON llega bien (_apply_remote_config) o **Fallback** si no hay red o viene mal formado (se queda con lo local).

Ver: remote_config.py: fetch(); main.py: _apply_remote_config()

P: ¿De dónde viene el valor remoto?

De **raw.githubusercontent.com/zikness/FirmApp/main/remote/config.json** (constante REMOTE_CONFIG_URL).

Ver: main.py

P: ¿Cómo se refleja en tiempo real?

Clock.schedule_interval vuelve a pedir el valor cada 5 s (REMOTE_CONFIG_POLL_INTERVAL). Para la demo sin hacer push, REMOTE_CONFIG_URL se apunta al archivo local: fetch() detecta que no es http(s) y lo lee del disco; al editar y guardar el JSON, la pantalla cambia sola.

Ver: main.py: _poll_remote_config(); remote_config.py: fetch()

P: ¿Por qué no usar solo GitHub en la demo?

Porque su CDN cachea el archivo varios minutos tras un push; un cambio 'en vivo' tardaría en verse.

Ver: README.md: Remote Config

P: ¿Qué problema técnico encontraste con la respuesta?

GitHub sirve los .json como **text/plain**, y UrlRequest solo decodifica automático si es application/json. Solución: **json.loads()** manual del texto recibido.

Ver: remote_config.py: _aplicar()

6. Detalles técnicos que pueden preguntar

P: ¿Cómo lograste el efecto de tarjeta flotante sobre fondo oscuro?

Se fotografía el Dashboard con **export_as_image()** y se pinta oscurecido detrás de la pantalla Firmar/Éxito. Se dibuja con **canvas.Rectangle** (SnapshotBackground) porque el widget Image renderizaba la textura diminuta; además se corrige el volteo con flip_vertical().

Ver: widgets.py: SnapshotBackground; main.py: _snapshot_home()

P: ¿Por qué los puntos de la contraseña son '•' y no '●'?

La fuente DM Sans no incluye el carácter ● (se veía un cuadro vacío); verifiqué con fonttools que sí incluye • (U+2022).

Ver: interfazrrhh.kv: password_mask

P: ¿Por qué el texto del campo RUT/correo está a la izquierda y no centrado?

Kivy deja de centrar un TextInput cuando el texto se acerca al ancho del campo y sigue al cursor. Alinear a la izquierda es estable y el estándar de formularios.

Ver: interfazrrhh.kv: LoginIdentifierInput

P: ¿Cómo resolviste la fuga de memoria del modal original?

Pressable hacía Window.bind(mouse_pos=...) sin desconectarlo nunca. Al pasar a ScreenManager y desvincularlo, se eliminó.

Ver: widgets.py: Pressable

7. Modificaciones en vivo (posibles pedidos del profesor)

P: Cambiar el texto del banner remoto.

Editar **banner_message** en remote/config.json (con la ruta local activa en REMOTE_CONFIG_URL) y guardar; en máx. 5 s cambia en pantalla.

Ver: remote/config.json

P: Mostrar más o menos documentos recientes.

Cambiar **max_recent_docs** en remote/config.json. Lo aplica dashboard.set_max_recent().

Ver: remote/config.json; main.py

P: Cambiar cada cuánto consulta la configuración.

Modificar la constante **REMOTE_CONFIG_POLL_INTERVAL** (segundos) en main.py.

Ver: main.py

P: Exigir una contraseña más larga.

En el método que habilita el botón (set_password_filled en LoginScreen / RegisterScreen) cambiar la condición de largo mínimo.

Ver: main.py: LoginScreen, RegisterScreen

P: Cambiar el color de la app.

Editar los colores en **theme.py**; el .kv y los widgets los leen desde ahí.

Ver: theme.py

8. Uso de IA y pregunta de cierre

P: ¿Qué hizo la IA y qué hiciste tú?

La IA (Claude Code) generó y corrigió código; yo definí el alcance, probé cada versión en mi computador, detecté bugs con capturas de pantalla (campo mal alineado, contraseña ilegible, correo imposible de escribir) y decidí las bases de diseño. Detalle en USO_IA.md, sección 5 (Ajustes).

Ver: USO_IA.md

P: ¿Cómo responde la solución a la necesidad diagnosticada?

La encuesta mostró que el trabajador quiere firmar rápido desde el celular. FirmApp deja ver lo pendiente al abrir, firmar con un toque (RUT o dibujo), confirma visualmente la firma, avisa lo urgente con notificaciones y recuerda lo firmado al reabrir la app.

Ver: FUNDAMENTACION-UX-UI.md